DressMe

Architecture des agents IA

Dossier de validation technique

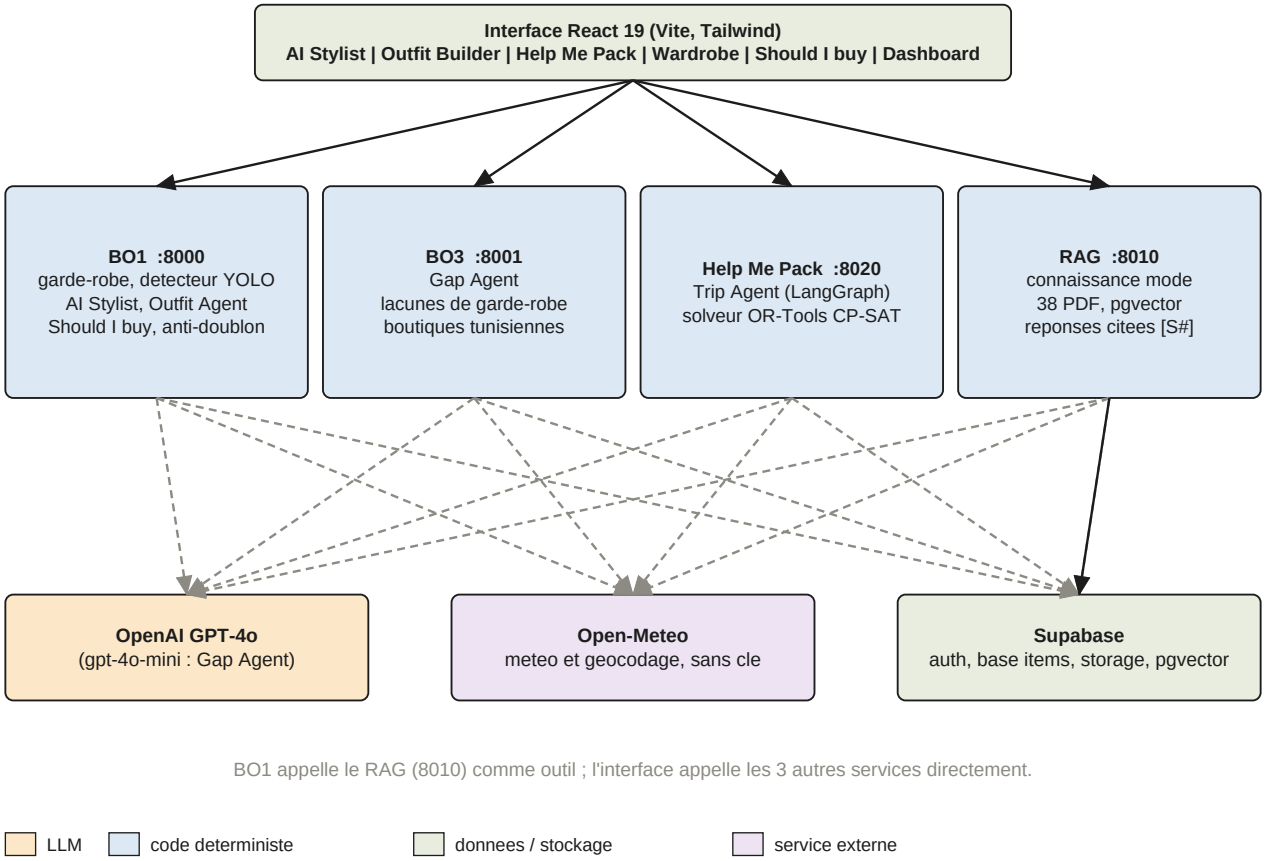
5 agents, 3 pipelines LLM, 1 RAG, 4 services

Graphes, prompts, outils, integration, garde-fous et limites

Projet de fin d'etudes - application DressMe (React, FastAPI, LangGraph, OpenAI, Supabase)

1. Vue d'ensemble

DressMe est une application React 19 qui parle a **quatre services FastAPI** (BO1, BO3, Help Me Pack, RAG). Supabase fournit l'authentification, la base de donnees (garde-robe) et le stockage des images. OpenAI fournit le LLM, Open-Meteo la meteo reelle.



BO1 appelle le RAG (8010) comme outil ; l'interface appelle les 3 autres services directement.

Figure 1 : architecture globale, services et dependances externes.

Principe commun a tous les agents : le LLM decide et redige, le code deterministe verifie et calcule. Aucun agent ne peut citer une piece absente de la garde-robe, inventer la meteo ou inventer une source.

Ports et lancement

Service	Port	Role	Lancement
BO1 (backend/app)	8000	garde-robe, detecteur, AI Stylist, Outfit Agent, Should I buy, avatar	start_bo1.ps1
BO3 (DressMe UI Blueprint/backend)	8001	Gap Agent, produits de boutiques tunisiennes	start_bo3.ps1
Help Me Pack (backend/main.py)	8020	Trip Agent et solveur CP-SAT	start_packing.ps1
RAG (rag_backend)	8010	connaissance de mode citee	start_rag.ps1
Interface (Vite)	8443	application React (https de developpement)	pnpm dev

2. Combien d'agents ?

5 agents sont appeles depuis l'interface. **3 pipelines** utilisent un LLM sans etre des agents (le LLM ne choisit pas ses outils). Une **pile separee de l'equipe** (Stylist BO4 et orchestrateur) existe aussi, non branchee a l'interface React.

#	Agent	Fonctionnalite	Technique	Fichier
1	AI Stylist	page AI Stylist	boucle d'appel d'outils GPT-4o, 3 outils	backend/app/agent.py
2	Outfit Builder Agent	page Outfit Builder	graphe LangGraph, 5 noeuds	backend/app/outfit_agent.py
3	Trip Agent	Help Me Pack	GPT-4o, 10 outils, LangGraph (2 graphes), CP-SAT	packing_backend/trip_agent/
4	Gap Agent (BO3)	Wardrobe Gaps, Shop Smart, Outfits to buy	5 outils, 6 etapes max, XGBoost	DressMe UI Blueprint/backend/agent/gap_agent.py
5	Agent anti-doublon	Live scan (camera)	1 outil (search_wardrobe)	backend/app/agent.py (check_duplicate)

Pipeline LLM	Role	Fichier
Catalogage vision	detecteur YOLO, puis GPT-4o vision (JSON strict) : categorie, couleur, matiere, saison, occasion	backend/app/enricher.py
Should I buy this?	detecteur + LLM, moteur d'achat (XGBoost + regles), puis LLM qui redige 3 sections	smart_purchasing/inference.py
Reponse RAG	recherche hybride, puis LLM qui repond uniquement avec les sources	rag_backend/rag/rag_service.py

Pile separee (non branchee a React)	Technique
Stylist BO4 (bo4/)	LangGraph a 9 noeuds, Ollama qwen2.5:3b, moteur de tenues heuristique (couleur 30, style 20, occasion 30, saison 20)
Orchestrateur (orchestrator/, port 8070)	LangGraph : route_intent (regles puis LLM local) vers stylist, garde-robe, achat, RAG ou clarification

LangGraph est utilise par l'Outfit Builder Agent, le Trip Agent, le Stylist BO4 et l'orchestrateur. L'AI Stylist, le Gap Agent et l'agent anti-doublon utilisent la boucle d'appel d'outils native d'OpenAI, ecrite a la main.

3. AI Stylist

Entree : `POST /api/agent/search` avec message, city, country, unit. La ville du profil est utilisee, sinon **Tunis, Tunisie**. **Modele :** GPT-4o, temperature 0,3, sortie JSON, 8 tours maximum.

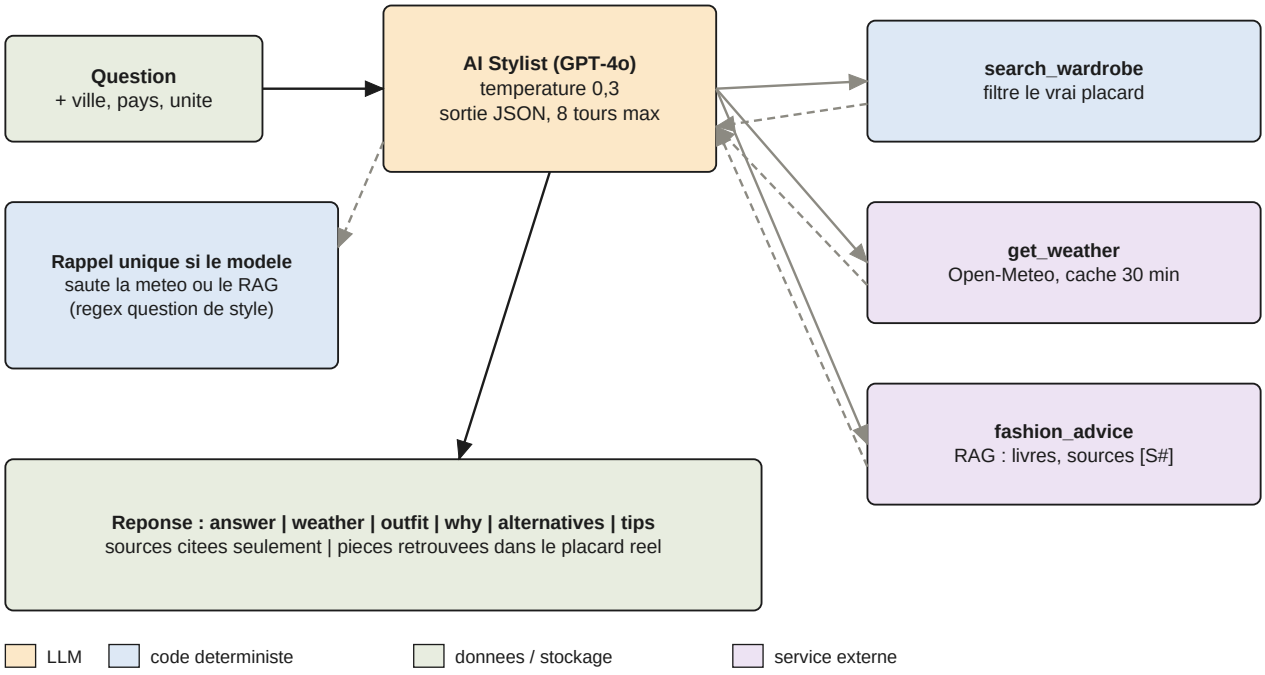


Figure 2 : boucle d'outils de l'AI Stylist.

Outils

Outil	Ce qu'il fait	Source
search_wardrobe	filtre la vraie garde-robe (categorie, couleur, saison, occasion, style, chaleur, texte)	table items (Supabase)
get_weather	prevision du jour voulu (aujourd'hui, demain, "vendredi"), en C ou F, 30 min de cache	Open-Meteo
fashion_advice	une question de style par sujet (jusqu'a 3), reponse avec [S#]	RAG (8010)

Prompt systeme (regles)

- Toujours appeler **search_wardrobe** avant de repondre ; ne parler que de pieces possedees ; si rien ne convient, le dire.
- **Meteo** : pour toute question "que porter / quelle tenue", appeler get_weather pour le jour voulu et citer les chiffres reels.
- **Livres** : pour toute question de style, appeler fashion_advice une fois par sujet (code vestimentaire, couleurs, tissus du climat).
- Ne jamais presenter un conseil de livre comme venant du placard ; ne jamais inventer de source.
- Position claire et convaincante : "My pick: ..." seulement pour une demande de conseil.

Format de reponse final (JSON, sans markdown ni emoji)

Cle	Contenu
answer	2 phrases : le choix puis la meilleure raison
weather	1 a 2 phrases : prevision chiffree et ce qu'elle impose
outfit	0 a 3 tenues, noms exacts des pieces reliees par " + "

Cle	Contenu
why	4 a 6 raisons : meteo, occasion, couleurs, matieres, role des pieces ; chacune appuyee sur un fait reel et un [S#]
alternatives	0 a 3 lignes : ce qui a ete envisage et pourquoi ecarte
tips	2 a 3 conseils pratiques, avec [S#] quand ils viennent des livres

Garde-fous

- Rappel unique si le modele saute la meteo ou le RAG sur une question qui les demande (detection par expression reguliere FR/EN).
- Seules les sources reellement citees dans le texte sont renvoyees ; les pieces de la tenue sont retrouvees par leur nom dans le placard.
- Ville inconnue du service meteo : essai sur les premiers mots du nom, sinon meteo de Tunis annoncee comme telle.
- RAG ou meteo en panne : l'agent repond sans eux et le dit.

4. Outfit Builder Agent (LangGraph)

Route `POST /api/outfits/agent` (occasion, saison, ville, tenues deja montrees). Un graphe d'etat partage ou chaque noeud a un seul travail ; le LLM n'intervient que ou le jugement est necessaire.

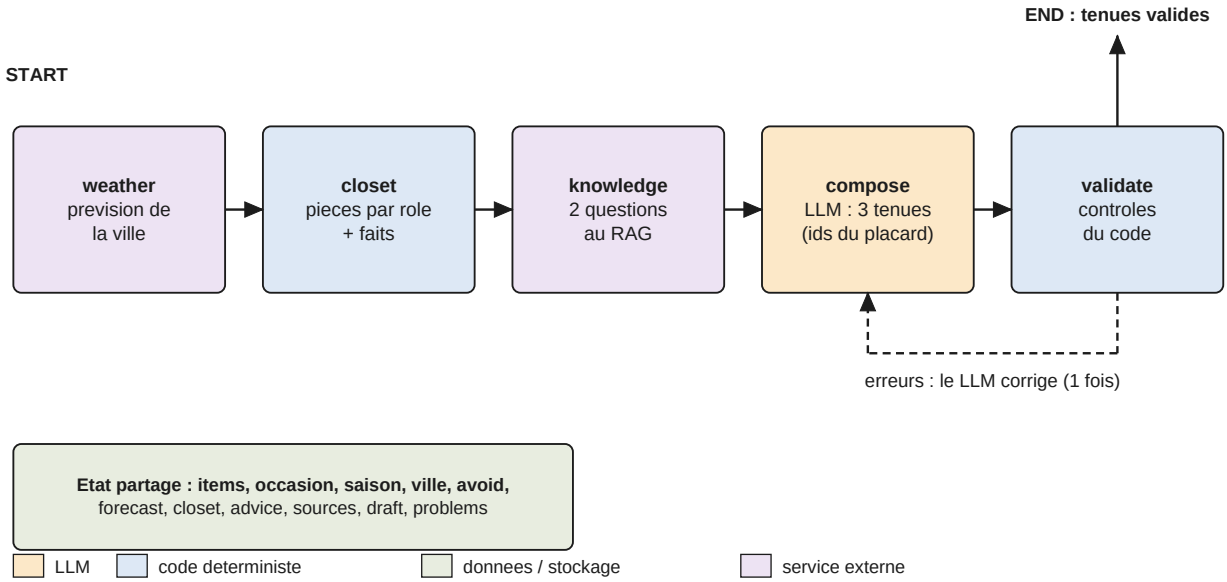


Figure 3 : graphe LangGraph de l'Outfit Builder.

Noeud	Role	LLM
weather	prevision reelle de la ville (Tunis par defaut)	non
closet	pieces classees par role : hauts, bas, robes, vestes, chaussures, avec couleur, matiere, saison, chaleur, formalite	non
knowledge	2 questions au RAG : code vestimentaire de l'occasion ; couleurs et tissus pour le climat du jour	RAG
compose	construit 3 tenues avec les identifiants du placard, donne un titre, une note meteo chiffree et 3 a 5 raisons	GPT-4o, JSON
validate	ids reels ; robe seule ou haut + bas ; chaussures ; pas deux pieces du meme role ; veste s'il fait froid ou pluvieux	non

Boucle de correction

Si une tenue est invalide, le LLM est renvoye **une seule fois** avec la liste exacte des erreurs ("Outfit 2: it has no shoes"). Seules les tenues valides sont retournees. Le bouton Shuffle envoie les tenues deja vues (`avoid`) pour en obtenir d'autres.

Interface et repli

- Bandeau meteo, titre de chaque look, phrase "Weather", liste "Why this outfit", bouton essayage virtuel, livres cites.
- Si l'agent est indisponible, l'ancien constructeur a regles (`outfitBuilder.ts`) prend le relais : la page affiche toujours quelque chose.
- Les pieces manquantes ("a rain jacket") ne sont signalees que si les conditions le justifient.

5. Trip Agent : Help Me Pack

Modele : GPT-4o, temperature 0, appel d'outils natif, 12 iterations maximum. Le LLM choisit l'outil suivant ; le code valide les arguments (Pydantic), impose les dependances et calcule la valise avec **OR-Tools CP-SAT**.

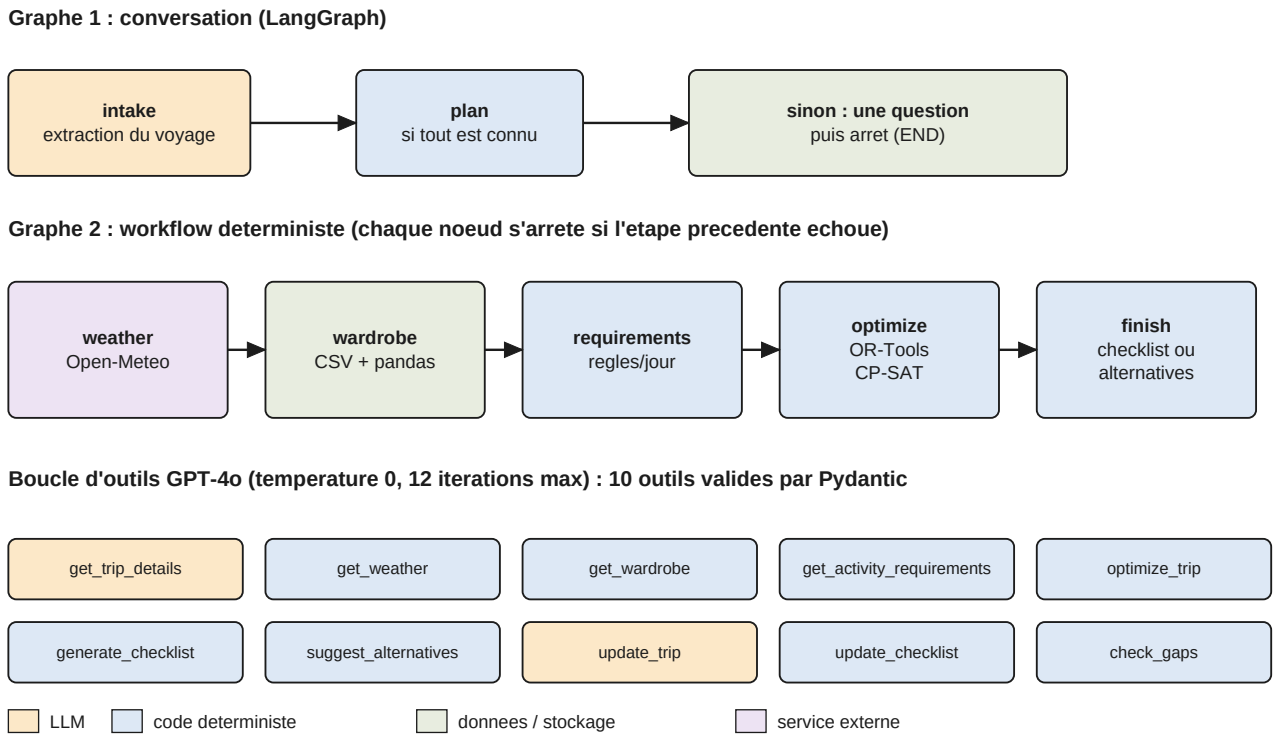


Figure 4 : deux graphes LangGraph et les 10 outils du Trip Agent.

Memoire

Type	Contenu	Duree
Travail	AgentState : voyage valide, meteo, placard, exigences, resultat du solveur, capsule, checklist, erreurs	une session
Conversation	16 derniers messages injectes dans le prompt	une session
Session	instance de l'agent par session_id, protegee par un verrou	jusqu'au redemarrage
Client	session_id dans le localStorage pour reprendre apres un rafraichissement	navigateur

Regles du prompt (10)

- Ne jamais inventer destination, dates, duree, activites, limite de bagage, meteo, pieces ni resultat du solveur.
- Voyage nouveau : get_trip_details d'abord ; s'il manque un champ, poser UNE question concise et s'arreter.
- Ordre : get_weather, get_wardrobe (une fois), get_activity_requirements, optimize_trip, puis generate_checklist.
- INFEASIBLE : suggest_alternatives ; ne jamais relacher silencieusement une contrainte dure.
- update_trip seulement pour modifier un voyage ; update_checklist pour cocher sans relancer le solveur.
- Une erreur d'outil se corrige ou s'explique, jamais un succes fabrique.

6. Gap Agent BO3 et boutiques tunisiennes

Modele : gpt-4o-mini par defaut, temperature 0,2, 6 etapes maximum. Il repond a "que manque-t-il dans ma garde-robe pour tel objectif ?" uniquement a partir des resultats d'outils.

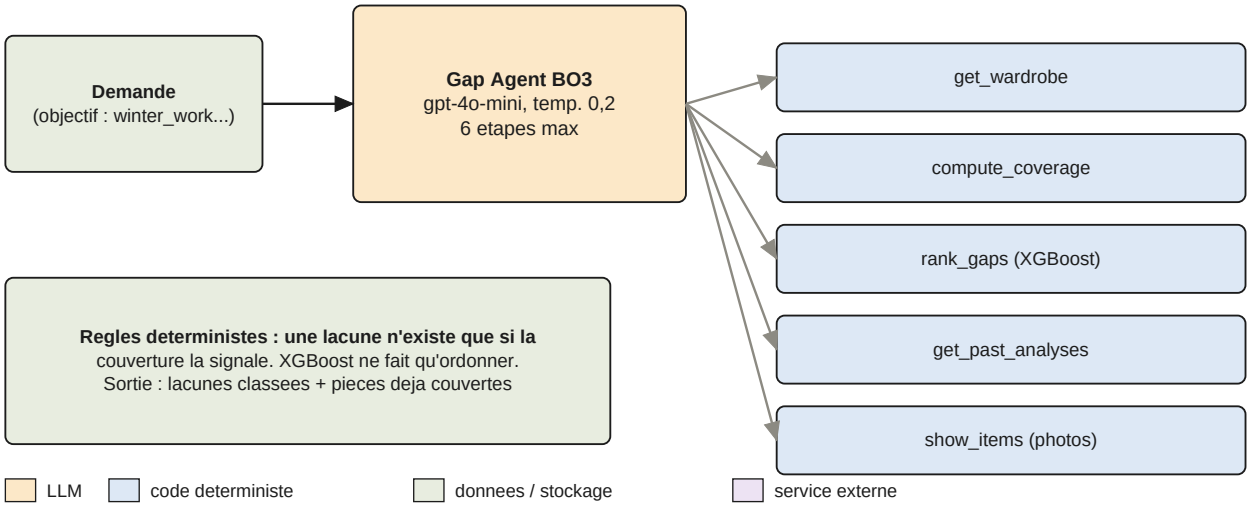


Figure 5 : Gap Agent et ses 5 outils.

- **Couverture** : regles deterministes par objectif et saison. Une lacune n'est reelle que si ces regles la signalent.
- **XGBoost** ordonne les lacunes ; son score n'est pas une probabilite, et il est entraine sur des donnees synthetiques tant qu'il n'y a pas de retours reels (le prompt le fait dire).
- **Departement** (femme, homme) : jamais de piece d'un autre departement ; jamais de genre devine.
- **Aval** : les lacunes alimentent "Outfits to buy" du tableau de bord, avec des produits de boutiques tunisiennes (recherche d'images ddgs) et l'essayage virtuel.
- Reponse dans la langue de l'utilisateur (anglais, francais, arabe tunisien).

Agent anti-doublon (Live scan)

Apres chaque capture camera, GPT-4o (temperature 0,1, 250 tokens) appelle `search_wardrobe` pour decider s'il s'agit du **meme** vetement scanne deux fois, et non d'"un autre t-shirt marine". Reponse JSON : duplicate, similar_item_id, note.

7. Pipelines assistes par LLM

7.1 Catalogage d'un vetement

Sans cle API ou en cas d'erreur : regles de secours (fallback_enrich)

Figure 6 : de la photo a la fiche vetement.

GPT-4o ne change la categorie du detecteur que si l'image le montre clairement ; la couleur est choisie dans la palette la plus proche de la mesure sur les pixels ; les scores de confiance doivent etre honnetes (bas si l'image est ambigue).

7.2 Should I buy this?

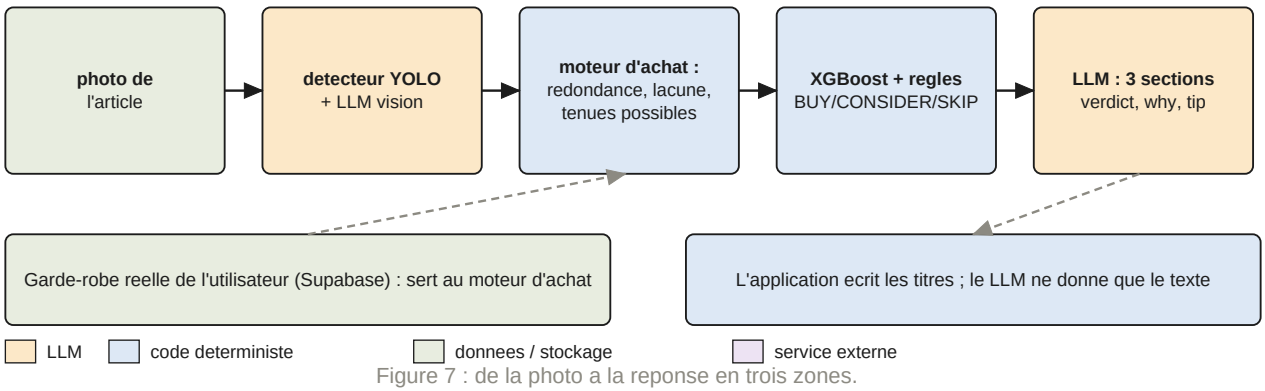


Figure 7 : de la photo a la reponse en trois zones.

- L'article est reconnu par le **meme** detecteur + LLM que le placard : pas de second moteur de vision.
- Le moteur d'achat calcule redondance, lacune comblee, tenues possibles avec les pieces possedees et prix par tenue.
- Le LLM (temperature 0,5) ne redige que trois sections : **verdict** ("Yes, buy it:", "Think twice:", "Skip it:" + raison), **why_it_works** (3 phrases avec un fait : nombre de tenues, nom d'une piece possedee, prix) et **final_advice**.
- Le prompt interdit le jargon (XGBoost, score, redondance), les emojis, le markdown et les symboles monetaires ; l'application ecrit les titres en gras.

8. Le RAG : connaissance de mode citee

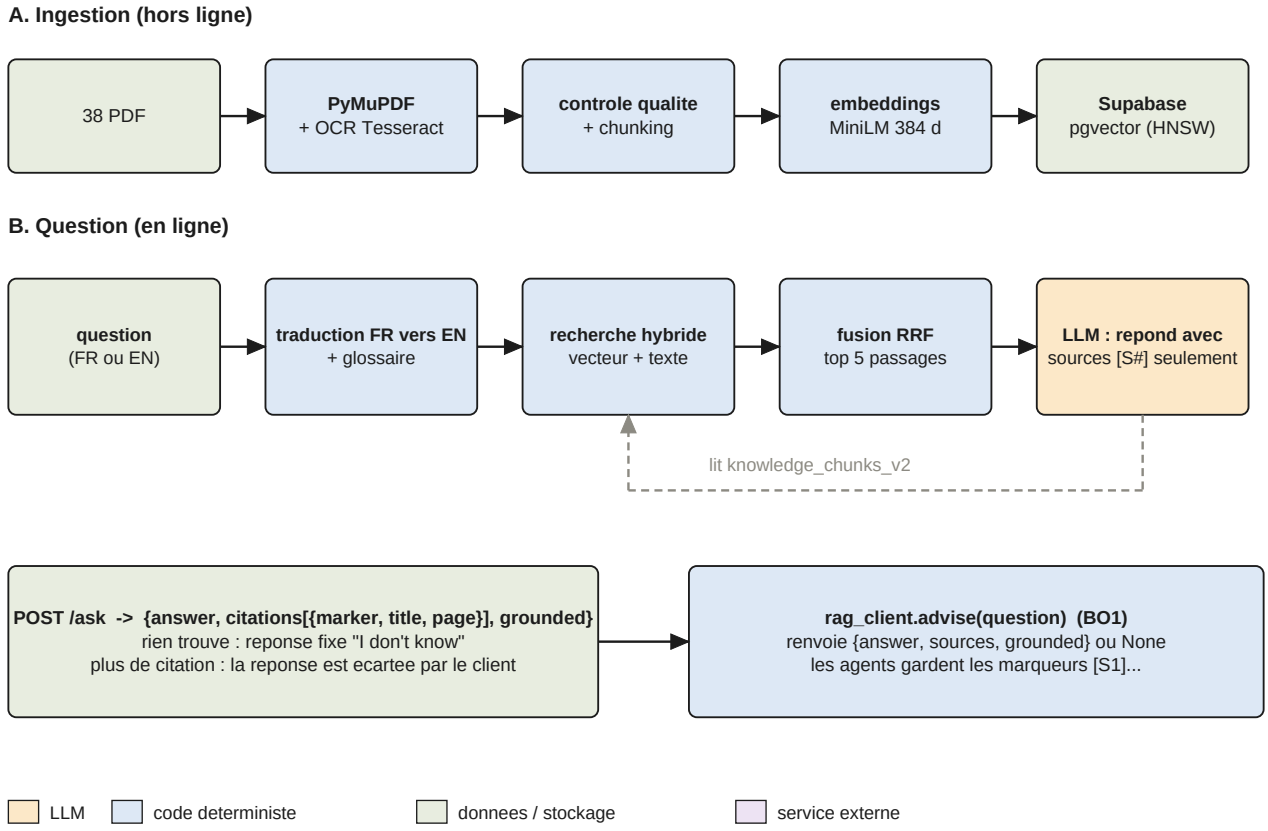


Figure 8 : ingestion et interrogation du RAG.

Element	Detail
Corpus	38 PDF (couleurs, tissus, codes vestimentaires, capsules, accessoires...), OCR Tesseract pour les livres scannes
Index	table knowledge_chunks_v2 (Supabase Postgres + pgvector, HNSW, cosinus), 384 dimensions, all-MiniLM-L6-v2
Recherche	hybride : vecteurs + recherche plein texte Postgres, fusion RRF, top 5 passages
Question	francais traduit en anglais (glossaire), anglais inchange
Generation	repond UNIQUEMENT avec les sources numerotees ; chaque puce finit par [S#] ; sinon la reponse fixe "I don't know"
API	POST /ask, /ask/stream (SSE), cle d'API, limitation de debit, historique de conversation

Integration dans les agents

- `rag_client.advise(question)` appelle /ask et renvoie {answer, sources:[{marker, title, page}], grounded}, ou None si le service est arrete, vide ou sans citation.
- L'AI Stylist l'expose comme outil **fashion_advice** ; l'Outfit Agent l'appelle dans le noeud **knowledge**.
- Les agents gardent les marqueurs [S#] dans leurs phrases ; l'interface les affiche en badges (survol : livre et page) et liste les livres sous "From fashion books".
- Le RAG ne connait pas le placard : c'est l'agent qui fait le lien entre la regle du livre et les pieces reelles.

9. Qui utilise quoi

Fonctionnalite	Agent / pipeline	Meteo	RAG	Placard	Solveur / ML
AI Stylist	AI Stylist	oui	oui	oui	non
Outfit Builder	Outfit Builder Agent	oui	oui	oui	non
Help Me Pack	Trip Agent	oui	hook prevu	oui	OR-Tools CP-SAT
Wardrobe Gaps, Shop Smart, Outfits to buy	Gap Agent	non	non	oui	regles + XGBoost
Should I buy this?	pipeline detecteur + LLM + moteur d'achat	non	non	oui	XGBoost
Live scan	detecteur + agent anti-doublon	non	non	oui	YOLO
Ajout d'un vetement	detecteur + catalogage vision	non	non	non	YOLO

Prompts : parametres LLM par composant

Composant	Modele	Temperature	Sortie	Limite
AI Stylist	GPT-4o	0,3	JSON	8 tours, 1800 tokens
Outfit Agent (compose)	GPT-4o	0,6	JSON	2 passes max, 2200 tokens
Trip Agent	GPT-4o	0	outils + JSON	12 iterations
Gap Agent	gpt-4o-mini	0,2	Markdown	6 etapes
Anti-doublon	GPT-4o	0,1	JSON	250 tokens
Should I buy (rédaction)	GPT-4o	0,5	JSON 3 sections	-
Catalogage vision	GPT-4o	-	schema JSON strict	-

Cycle d'une requete (exemple : "Que mettre demain pour un diner ?")

- 1. React envoie message + ville + unite du profil a BO1.
- 2. L'AI Stylist appelle get_weather (demain), search_wardrobe (saison, occasion) puis fashion_advice (code vestimentaire, couleurs).
- 3. GPT-4o redige le JSON : choix, meteo chiffree, tenue, 4 a 6 raisons, alternatives, conseils.
- 4. BO1 filtre les sources citees, retrouve les pieces dans le placard et renvoie la reponse + la prevision.
- 5. React affiche les zones Weather, Outfit idea (photos), Why I chose this, What else I considered, Tips, From fashion books.

10. Garde-fous, securite et tests

Theme	Mesure
Ancrage	identifiants du placard seulement ; outils en lecture ; sources numerotees ; meteo reelle ; "never invent" dans chaque prompt
Validation par du code	Outfit Agent (regles de composition), Trip Agent (Pydantic + solveur), AI Stylist (filtrage des sources citees)
Degradation douce	RAG, meteo ou LLM indisponibles : reponse partielle ou repli par regles, jamais une page cassee
Securite	jeton Supabase verifie sur chaque route ; chaque utilisateur ne voit que son placard ; cles API cote serveur uniquement ; limitation de debit du RAG
Honnetete	le prompt du Gap Agent dit que le classeur est entraine sur des donnees synthetiques ; le RAG repond "I don't know" plutot que d'inventer
Tests	85 tests backend (faux LLM, faux RAG, faux service meteo), 30 tests frontend, tests du moteur d'achat et du Trip Agent ; essais reels de bout en bout avec GPT-4o, le RAG et Open-Meteo

Limites assumees

- Le RAG ne connait pas le placard de l'utilisateur ; il ne contient que des livres de mode.
- Le classeur XGBoost du Gap Agent est entraine sur des donnees synthetiques jusqu'a l'arrivee de retours reels.
- L'AI Stylist n'a pas de memoire entre deux messages ; la memoire du Trip Agent est en memoire serveur (perdue au redemarrage).
- Le Stylist BO4 et l'orchestrateur LangGraph ne sont pas encore relies a l'interface React.
- La meteo est celle d'un jour (jusqu'a 15 jours) pour une seule ville.
- Pistes : memoire de preferences par utilisateur, journal de ce qui est porte, protocole A2A entre agents, migration de l'AI Stylist vers LangGraph.